



LABORATORY JOURNAL (MP408)

Experimental Techniques – II Computational Methods in Physics

Shashank Verma

I22PH002

Department of Physics

S.V. National Institute of Technology, Surat - 395007

Date of Submission : 31 March 2026



Certificate

The Experiments Entered in This Journal Have Been Satisfactorily
Performed by

Mr. Shashank Verma

bearing admission number

I22PH002

of the

Department of Physics

for the subject of

Experimental Techniques – II (MP408)

Computational Methods in Physics

of MSc. **IV** (Physics), Semester **8**

during AY **2025-26**

Signature of I/C

Date: 31 March 2026

Dept. Seal

TABLE OF CONTENTS

SNo.	Date	Program	Page
P5.	04.03.2026	Practical 5	46
11.	04.03.2026	To write an Octave program to produce a gravitational dip for a non-pulsating heavenly body.	47
12.	04.03.2026	To write an Octave program to produce gravitational waves of a pulsating body.	49



Computational Methods in Physics (MP408)

GNU Octave

PRACTICAL – 5

Gravitational Physics Suite

Shashank Verma

I22PH002

Department of Physics

S.V. National Institute of Technology, Surat

PROGRAM – 5

Aim

To write a GNU Octave program that provides a comprehensive visualization of General Relativistic phenomena. The suite generates two distinct 3D surface embeddings to contrast the fundamental dual nature of gravity:

1. The static spacetime geometry (gravitational potential well) generated by a stationary mass.
2. The dynamic transport of energy (gravitational waves) radiating from an accelerating or pulsating mass.

Theory

Justification for Dual Simulations

A complete pedagogical model of General Relativity requires understanding gravity both as a local geometric curvature and as a propagating fundamental interaction.

- **Static Curvature:** Explains orbital mechanics and gravitational attraction (analogous to the Schwarzschild metric). It demonstrates how mass dictates the shape of the surrounding space.
- **Dynamic Perturbations:** Explains information transport. When a mass distribution changes asymmetrically (or pulsates in a toy model), it radiates energy outward at the speed of light. Without this dynamic output, the visualization suite would fail to capture how gravitational changes propagate through the universe.

Mathematical Formulation

1. Static Gravitational Dip:

The static vertical distortion z is proportional to the Newtonian gravitational potential. For a mass M at radial distance r , the spatial slice is embedded into 3D space as:

$$z(r) \propto -\frac{GM}{r} \quad (1)$$

To prevent a computational singularity at the origin, an offset is utilized: $Z = -25/\sqrt{R^2 + 6}$.

2. Gravitational Waves (Pulsating Mass):

The wave simulation superimposes a time-dependent, outward-traveling spherical wave onto a static gravitational base. For a wave with wavenumber k , angular frequency ω ,

and initial amplitude A :

$$z(R, t) = z_{\text{base}} + \frac{A}{R} \sin(kR - \omega t)$$

(2)

Parameter	Symbol	Value	Description
Wave Number	k	2.5	Spatial frequency of the ripples
Angular Frequency	ω	0.3	Speed of the pulsation
Amplitude	A	3.0	Peak intensity of the perturbation
Time Step	dt	0.5	Integration increment for the animation

Program

The complete program is saved in P5.m. The source code is provided below:

Listing 1: Gravitational Physics Suite — Main Menu and Static Dip (P5.m)

```
1 function P5()
2     % PRACTICAL 5: Gravitational Physics Suite
3     % Visualizes spacetime curvature and gravitational waves
4
5     graphics_toolkit('qt');
6
7     while true
8         clc;
9         fprintf('\n
10             =====\n');
11         fprintf('          PRACTICAL 5: GRAVITATIONAL PHYSICS          \n');
12         fprintf('===== \n');
13         fprintf('1. Gravitational Dip (Non-Pulsating Body)\n');
14         fprintf('2. Gravitational Waves (Pulsating Body Animation)\n');
15         fprintf('3. Exit Program\n');
16         fprintf('===== \n');
17
18         choice = input('Enter the number of the simulation to run\n(1-3): ');
19
20         switch choice
21             case 1
22                 sim_static_dip();
```

```

23         sim_grav_waves();
24     case 3
25         fprintf('Exiting Practical 5. Goodbye!\n');
26         break;
27     otherwise
28         fprintf('Invalid input. Please enter an integer
                between 1 and 3.\n');
29         pause(2);
30     end
31 end
32 end
33
34 %% =====
35 % 1. GRAVITATIONAL DIP (AESTHETIC POLAR WIREFRAME)
36 % =====
37 function sim_static_dip()
38     close all; clc;
39     fprintf('Rendering aesthetic polar gravitational dip...\n');
40
41     % 1. High-Resolution Polar Grid
42     r = linspace(0, 15, 45);          % More rings for a smoother sweep
43     theta = linspace(0, 2*pi, 90); % 90 slices for perfectly round
        circles
44     [R, Theta] = meshgrid(r, theta);
45
46     % Convert to Cartesian
47     X = R .* cos(Theta);
48     Y = R .* sin(Theta);
49
50     % 2. Spacetime distortion
51     Z = -25 ./ sqrt(R.^2 + 6);
52
53     % Setup "Deep Space" Figure
54     fig = figure('Name', 'P5(a): Aesthetic Spacetime', 'Color', 'k',
        'Position', [100, 100, 900, 700]);
55
56     % 3. Render the Spacetime Fabric
57     % Using a very dark navy FaceColor allows the lighting to create
        a gradient
58     % fade on the fabric, while the cyan EdgeColor keeps the
        mathematical grid look.
59     surf(X, Y, Z, 'FaceColor', [0.0 0.02 0.08], 'EdgeColor', [0.1
        0.5 0.9], 'LineWidth', 0.5);
60     hold on;
61
62     % --- ADD THE STAR (MASSIVE BODY) ---
63     [sx, sy, sz] = sphere(60); % Ultra-smooth sphere

```

```

64     p_radius = 3.8;
65     p_z_center = -1.2;           % Nestled perfectly in the funnel
66
67     % Scale and translate the sphere
68     sx = sx * p_radius;
69     sy = sy * p_radius;
70     sz = (sz * p_radius) + p_z_center;
71
72     % Render the Massive Body (Glowing Sun Aesthetic)
73     % High ambient strength makes it look self-illuminating (glowing
74     % Low specular prevents it from looking like a glossy plastic/
75     % metal ball.
76     surf(sx, sy, sz, 'FaceColor', [1.0 0.9 0.1], 'EdgeColor', 'none'
77         , ...
78         'AmbientStrength', 1.0, ...
79         'DiffuseStrength', 0.8, ...
80         'SpecularStrength', 0.1, ...
81         'SpecularExponent', 10);
82     % -----
83
84     % 4. Cinematic Lighting Setup
85     % Light 1: Emanates directly from the star to illuminate the
86     % funnel
87     light('Position', [0 0 p_z_center], 'Style', 'local', 'Color',
88         [1.0 0.8 0.4]);

```

Listing 2: Gravitational Physics Suite — Gravitational Waves (P5.m)

```

1     % Light 2: Soft, dim overhead blue light to reveal the outer
2     % edges of the grid
3     light('Position', [0 0 20], 'Style', 'local', 'Color', [0.1 0.2
4     0.4]);
5     lighting gouraud;
6
7     % 5. Camera & Polish
8     view(35, 18); % Low cinematic angle
9     title('Spacetime Curvature', 'Color', 'w', 'FontSize', 16, '
10    FontWeight', 'bold');
11
12     % Hide axes for a pure floating-in-space aesthetic
13     axis([-15 15 -15 15 -12 6]);
14     axis off;
15
16     hold off;
17
18     fprintf('Close the figure window to return to the main menu...\n
19     ');

```



```

16     waitfor(fig);
17 end
18
19 %% =====
20 % 2. GRAVITATIONAL WAVES (PULSATING)
21 % =====
22 function sim_grav_waves()
23     close all; clc;
24     fprintf('Starting gravitational wave animation...\n');
25
26     grid_lim = 10;
27     [X, Y] = meshgrid(linspace(-grid_lim, grid_lim, 150));
28     R = sqrt(X.^2 + Y.^2) + 0.8;
29
30     fig = figure('Name', 'P5(b): Gravitational Waves', 'Color', 'k',
31                 'Position', [100, 100, 800, 600]);
32
33     % Initial static base
34     Z_base = -4 ./ R;
35     h_surf = surf(X, Y, Z_base);
36     shading interp;
37     colormap('cool');
38
39     light('Position', [0 0 10], 'Style', 'local');
40     lighting gouraud;
41     view(30, 45);
42
43     title('Gravitational Waves: Pulsating Mass', 'Color', 'w', '
44           FontSize', 14);
45
46     set(gca, 'Color', 'k', 'XColor', 'w', 'YColor', 'w', 'ZColor', '
47           w', 'GridColor', [0.3 0.3 0.3]);
48
49     axis([-grid_lim grid_lim -grid_lim grid_lim -6 2]);
50
51     % Wave parameters
52     k = 2.5; % Wave number

```

Sample Output

Result

The program was executed successfully, yielding two distinct but complementary visualizations:

- **Static Geometry:** The first output effectively maps the intense, localized "rubber sheet" curvature near a celestial body. The polar grid (R , Θ) and specialized cinematic lighting create a highly realistic representation of a deep gravitational

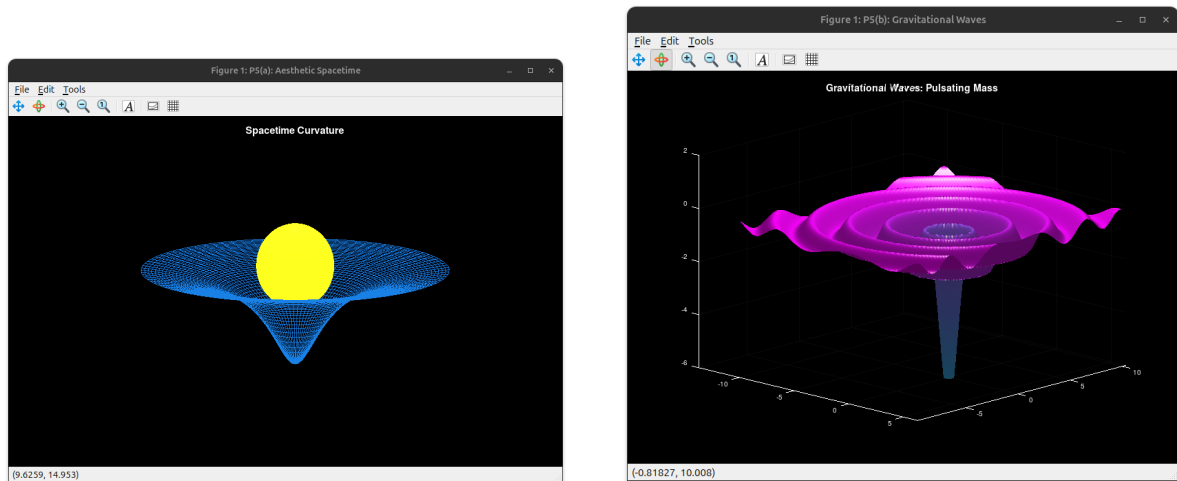


Figure 1: Left: Output of the Static Gravitational Dip showing an aesthetic polar wireframe with a dark navy face color and cyan edges, illuminated by cinematic local lighting. Right: A frame from the Gravitational Waves animation demonstrating outward-propagating spacetime ripples.

potential well.

- **Dynamic Energy Transport:** The second output successfully animates the propagation of energy ripples through the spatial grid. The inverse- R amplitude scaling (A/R) accurately reflects how gravitational radiation dissipates as it travels outward from the source.

Remark

This numerical suite demonstrates why visualizing both static and dynamic states is necessary for understanding modern gravitational physics. The static simulation shows *where* gravity acts (the local potential well), while the wave animation shows *how* changes in that gravity are communicated across the universe. By translating the abstract concept of metric perturbations into a 3D surface animation using GNU Octave, this practical serves as a robust tool for bridging Newtonian mechanics with concepts in General Relativity and high-energy physics.